

RETAILMART

SALES & CUSTOMER DATA ANALYSIS

presented by
Hazra Shaikh



**20
26**

SQL PROJECT

PROJECT OVERVIEW

RetailMart SQL Project is a beginner-to-intermediate database project that simulates a real retail business. It uses 5 related tables—Customers, Employees, Products, Orders, and OrderDetails—to analyze sales, customers, products, and employee performance using SQL. The project covers core SQL concepts including SELECT, WHERE, GROUP BY, HAVING, JOINS, Subqueries, and CASE for business reporting and data analysis.

Tech Stack: MySQL Workbench • SQL • CSV Data Import



DATABASE OVERVIEW – 5 RELATED TABLES

RetailMart's data is organized into five related tables:

- Customers: People who place orders — name, city, state, contact details, and the date they joined.
- Employees: RetailMart's sales and support staff, including their department, designation, salary, and manager.
- Products: Items RetailMart sells, with category, price, and current stock quantity.
- Orders: One row per order — which customer placed it, which employee handled it, when, where it shipped, and its status.
- OrderDetails: One row per product within an order — this is what lets a single order contain multiple products.

HOW THE TABLES RELATE TO EACH OTHER

```
Customers (1) -----< (Many) Orders
Employees (1) -----< (Many) Orders
Orders (1) -----< (Many) OrderDetails
Products (1) -----< (Many) OrderDetails
Employees (1) -----< (Many) Employees [ManagerID, self-referencing]
```



PROJECT QUESTIONS

Section 1 — Basic Retrieval & Filtering

Q1. List the name and city of every customer from the state of Maharashtra.

```
SELECT Customer_name, City  
FROM Customers  
WHERE state = 'Maharashtra';
```

	Customer_name	city
▶	Rohit Sharma	Pune
	Sneha Kulkarni	Mumbai
	Neha Joshi	Pune
	Arjun Desai	Pune
	Rajesh Kulkarni	Nagpur



PROJECT QUESTIONS

Q2. List all products priced above 1000, sorted from most to least expensive.

```
SELECT ProductName , Price  
FROM products  
where price > 1000  
ORDER BY Price DESC;
```

	ProductName	Price
▶	Office Chair	4999
	Study Table	3499
	Running Shoes	2499
	Bluetooth Headphones	1999
	Backpack	1299
	Whiteboard	1099



PROJECT QUESTIONS

Q3. Display the Order ID, order date, and status of every order shipped to Pune.

```
SELECT OrderID, OrderDate, status  
FROM Orders  
WHERE Shipcity='Pune' ;
```

	OrderID	OrderDate	status
▶	1	2023-06-01	Delivered
	6	2023-06-10	Delivered
	7	2023-06-12	Pending
	13	2023-06-27	Delivered
	19	2023-07-12	Delivered
●	NULL	NULL	NULL



PROJECT QUESTIONS

Q4. Find all employees in the Sales department who earn more than 40000.

```
SELECT  
Employee_Name,department,Salary  
FROM Employees  
WHERE Department = 'sales' AND Salary >  
40000;
```

	Employee_Name	department	Salary
▶	Ramesh Gupta	Sales	65000
	Kavita Menon	Sales	45000
	Sanjay Patil	Sales	42000
	Farah Sheikh	Sales	43000



PROJECT QUESTIONS

Q5. List the name, email, and join date of customers who joined after 1st March 2023.

```
SELECT Customer_name, Email, Joindate  
FROM Customers  
WHERE Joindate > '2023-03-01';
```

	Customer_name	Email	Joindate
►	Priya Nair	priya.nair@mail.com	2023-03-05
	Neha Joshi	neha.joshi@mail.com	2023-04-01
	Vikram Singh	vikram.singh@mail.com	2023-03-12
	Anjali Rao	anjali.rao@mail.com	2023-05-09
	Meera Pillai	meera.pillai@mail.com	2023-06-14
	Arjun Desai	arjun.desai@mail.com	2023-04-25
	Divya Kapoor	divya.kapoor@mail.com	2023-05-30
	Rajesh Kulkarni	rajesh.kulkarni@mail.com	2023-07-01
	Isha Bansal	isha.bansal@mail.com	2023-07-10



PROJECT QUESTIONS

Section 2 — Aggregate Functions & Grouping

Q6. Find the total number of orders placed by each customer.

```
SELECT CustomerId,  
COUNT(*) AS TotalOrders  
FROM Orders  
GROUP BY CustomerId;
```

	CustomerId	TotalOrders
▶	1	3
	2	2
	3	2
	4	2
	5	2
	6	1
	7	2
	8	1
	9	2
	10	1
	11	1
	12	1



PROJECT QUESTIONS

Q7. Find the total quantity sold for each product, ranked from highest to lowest.

```
SELECT ProductId,  
SUM(Quantity) AS Total_Quantity  
FROM Orderdetails  
GROUP BY ProductId  
ORDER BY Total_Quantity DESC;
```

	ProductId	Total_Quantity
▶	8	11
	11	11
	5	10
	1	6
	2	4
	3	4
	6	4
	7	4
	9	4
	4	2
	10	2
	12	1



PROJECT QUESTIONS

Q8. Find the average salary of employees in each department.

```
SELECT Department,  
AVG(Salary) AS Average_Salary  
FROM Employees  
GROUP BY Department;
```

	Department	Average_Salary
▶	Sales	48750.0000
	Support	44000.0000



PROJECT QUESTIONS

Q9. Find how many orders fall into each status (Delivered, Pending, Cancelled).

```
SELECT status ,SUM(OrderId) AS  
Total_Orders  
FROM Orders  
GROUP BY Status;
```

	status	Total_Orders
▶	Delivered	174
	Cancelled	15
	Pending	21



PROJECT QUESTIONS

Q10. Find product categories whose average price is above 1000, using HAVING.

```
SELECT Category, AVG(Price) AS AveragePrice  
FROM Products  
GROUP BY Category  
HAVING AveragePrice > 1000;
```

	Category	AveragePrice
▶	Electronics	1165.6667
	Furniture	4249.0000
	Fitness	1649.0000



PROJECT QUESTIONS

Section 3 — Working with Joins

Q11. Display the Order ID, customer name, and order date for every order.

```
SELECT O.OrderId,  
       C.Customer_Name, O.OrderDate  
FROM Orders O  
JOIN Customers C  
ON O.CustomerId = C.CustomerID;
```

	OrderId	Customer_Name	OrderDate
▶	1	Rohit Sharma	2023-06-01
	2	Sneha Kulkarni	2023-06-02
	3	Aman Verma	2023-06-03
	4	Priya Nair	2023-06-05
	5	Karan Mehta	2023-06-07
	6	Rohit Sharma	2023-06-10
	7	Neha Joshi	2023-06-12
	8	Vikram Singh	2023-06-15
	9	Anjali Rao	2023-06-18
	10	Suresh Iyer	2023-06-20
	11	Sneha Kulkarni	2023-06-22
	12	Meera Pillai	2023-06-25
	13	Arjun Desai	2023-06-27
	14	Aman Verma	2023-07-01
	15	Divya Kapoor	2023-07-03
	16	Priya Nair	2023-07-05
	17	Karan Mehta	2023-07-08
	18	Suresh Iyer	2023-07-10
	19	Rohit Sharma	2023-07-12
	20	Vikram Singh	2023-07-15



PROJECT QUESTIONS

Q12. List every order along with the name of the employee who handled it.

```
SELECT O.OrderId , E.Employee_Name  
FROM Orders O  
JOIN Employees E  
ON O.EmployeeId = E.EmployeeId;
```

	OrderId	Employee_Name
▶	1	Kavita Menon
	3	Kavita Menon
	6	Kavita Menon
	8	Kavita Menon
	11	Kavita Menon
	14	Kavita Menon
	17	Kavita Menon
	20	Kavita Menon
	2	Sanjay Patil
	5	Sanjay Patil
	9	Sanjay Patil
	12	Sanjay Patil
	15	Sanjay Patil
	18	Sanjay Patil
	4	Farah Sheikh
	7	Farah Sheikh
	10	Farah Sheikh
	13	Farah Sheikh
	16	Farah Sheikh
	19	Farah Sheikh



PROJECT QUESTIONS

Q13. Find the total revenue generated by each product (Quantity x Unit Price), highest first.

```
SELECT P.ProductName ,SUM(Quantity *  
UnitPrice) AS Total_Revenue  
FROM Products P  
JOIN OrderDetails O  
ON P.ProductId = O.ProductId  
GROUP BY ProductName  
ORDER BY Total_Revenue DESC;
```

	ProductName	Total_Revenue
►	Office Chair	19996
	Bluetooth Headphones	7996
	Study Table	6998
	Backpack	5196
	Running Shoes	4998
	Water Bottle	3839
	LED Desk Lamp	3596
	Wireless Mouse	3594
	Yoga Mat	3196
	Coffee Mug	2739
	Notebook Set	1990
	Whiteboard	1099



PROJECT QUESTIONS

Q14. Find the total bill amount for every order, along with the customer's name.

```
SELECT O.OrderId,C.Customer_name,  
SUM(od.Quantity * od.UnitPrice) AS  
Bill_Amount  
FROM Orders O  
JOIN Customers C  
ON C.CustomerId = O.CustomerId  
JOIN OrderDetails Od  
ON od.OrderId = O.OrderId  
GROUP BY OrderId, C.customer_name;
```

	OrderId	Customer_name	Bill_Amount
▶	1	Rohit Sharma	1795
	6	Rohit Sharma	2046
	19	Rohit Sharma	2243
	2	Sneha Kulkarni	2497
	11	Sneha Kulkarni	1099
	3	Aman Verma	4999
	14	Aman Verma	599
	4	Priya Nair	799
	16	Priya Nair	1197
	5	Karan Mehta	3194
	17	Karan Mehta	2598
	7	Neha Joshi	2499
	8	Vikram Singh	4494
	20	Vikram Singh	8498
	9	Anjali Rao	3395
	10	Suresh Iyer	5297
	18	Suresh Iyer	2898
	12	Meera Pillai	1597
	13	Arjun Desai	9998
	15	Divya Kapoor	3495



PROJECT QUESTIONS

Q15. Find all customers who have never placed an order.

```
SELECT C.Customer_Name, O.orderId
FROM Customers C
LEFT JOIN Orders O
ON C.CustomerId = O.CustomerId
WHERE O.OrderId IS NULL;
```

	Customer_Name	orderId
▶	Rajesh Kulkarni	NULL
	Isha Bansal	NULL



PROJECT QUESTIONS

Section 4 — Subqueries, CASE & Advanced Logic

Q16. Find customers who have placed more orders than the average number of orders per customer.

```
SELECT c.CustomerID,  
       c.Customer_Name,  
       COUNT(o.OrderID) AS TotalOrders  
FROM Customers c  
JOIN Orders o  
ON c.CustomerID = o.CustomerID  
GROUP BY c.CustomerID, c.Customer_Name  
HAVING COUNT(o.OrderID) > (  
    SELECT AVG(OrderCount)  
    FROM (  
        SELECT CustomerID,  
               COUNT(OrderID) AS OrderCount  
        FROM Orders  
        GROUP BY CustomerID  
    ) AS AvgOrders  
);
```

	CustomerID	Customer_Name	TotalOrders
▶	1	Rohit Sharma	3
	2	Sneha Kulkarni	2
	3	Aman Verma	2
	4	Priya Nair	2
	5	Karan Mehta	2
	7	Vikram Singh	2
	9	Suresh Iyer	2



PROJECT QUESTIONS

Q17. Find the employee who has processed the highest number of orders.

```
SELECT E.Employee_Name,  
       COUNT(O.orderID) AS TotalOrders  
FROM Employees E  
JOIN Orders O  
ON O.EmployeeId = E.EmployeeId  
GROUP BY e.Employee_Name  
ORDER BY TotalOrders DESC  
LIMIT 1;
```

	Employee_Name	TotalOrders
▶	Kavita Menon	8



PROJECT QUESTIONS

Q18. Classify every order's total value as High Value (>5000), Medium Value (2000-5000), or Low Value (<2000) using CASE.

```
SELECT o.OrderID,  
SUM(od.Quantity * od.UnitPrice) AS TotalValue,  
CASE  
    WHEN SUM(od.Quantity * od.UnitPrice) >  
5000 THEN 'HIGH VALUE'  
    WHEN SUM(od.Quantity * od.UnitPrice) BETWEEN  
2000 AND 5000 THEN 'HIGH VALUE'  
    ELSE 'LOW'  
END AS Ordercategory  
FROM Orders O  
JOIN OrderDetails od  
ON o.OrderID = od.OrderID  
GROUP BY o.OrderID;
```

	OrderID	TotalValue	Ordercategory
▶	1	1795	LOW
	2	2497	HIGH VALUE
	3	4999	HIGH VALUE
	4	799	LOW
	5	3194	HIGH VALUE
	6	2046	HIGH VALUE
	7	2499	HIGH VALUE
	8	4494	HIGH VALUE
	9	3395	HIGH VALUE
	10	5297	HIGH VALUE
	11	1099	LOW
	12	1597	LOW
	13	9998	HIGH VALUE
	14	599	LOW
	15	3495	HIGH VALUE
	16	1197	LOW
	17	2598	HIGH VALUE
	18	2898	HIGH VALUE
	19	2243	HIGH VALUE
	20	8498	HIGH VALUE



PROJECT QUESTIONS

Q19. Find all products that have never appeared in any order.

```
SELECT p.ProductID,  
       p.ProductName  
FROM Products p  
WHERE NOT EXISTS (  
    SELECT productId  
    FROM OrderDetails od  
    WHERE p.ProductID = od.ProductID  
);
```

	ProductID	ProductName
▶	13	Desk Organizer
●	NULL	NULL



PROJECT QUESTIONS

Q20. Find the top 3 customers by total amount spent, counting only Delivered orders.

```
SELECT C.CustomerID,  
       C.Customer_Name,  
       (  
         SELECT SUM(od.Quantity * od.UnitPrice)  
         FROM Orders O  
         JOIN OrderDetails od  
           ON o.OrderID = od.OrderID  
         WHERE o.CustomerID = c.CustomerID  
           AND o.Status = 'Delivered'  
       ) AS TotalSpent  
FROM Customers C  
ORDER BY TotalSpent DESC  
LIMIT 3;
```

	CustomerID	Customer_Name	TotalSpent
▶	7	Vikram Singh	12992
	11	Arjun Desai	9998
	9	Suresh Iyer	8195





THANK YOU!

